EE446
LAB 8
REPORT

Part One

Link to edge impulse project: https://studio.edgeimpulse.com/public/1008764/live

Idle state identification

```
Starting inferencing in 2 seconds...
Sampling...
Timing: DSP 83 ms, inference 540 us, anomaly 0 ms, postprocessing 49 us
#Classification predictions:
  idle: 0.996094
  lift: 0.000000
Starting inferencing in 2 seconds...
Sampling...
Timing: DSP 83 ms, inference 6 ms, anomaly 0 ms, postprocessing 49 us
#Classification predictions:
  idle: 0.996094
  lift: 0.000000
```

Lift state identification

```
#Classification predictions:
  idle: 0.000000
  lift: 0.996094
Starting inferencing in 2 seconds...
Sampling...
Timing: DSP 79 ms, inference 539 us, anomaly 0 ms, postprocessing 49 us
#Classification predictions:
  idle: 0.000000
  lift: 0.996094
```

Part Two

Question 1: Model architecture and ensemble flow

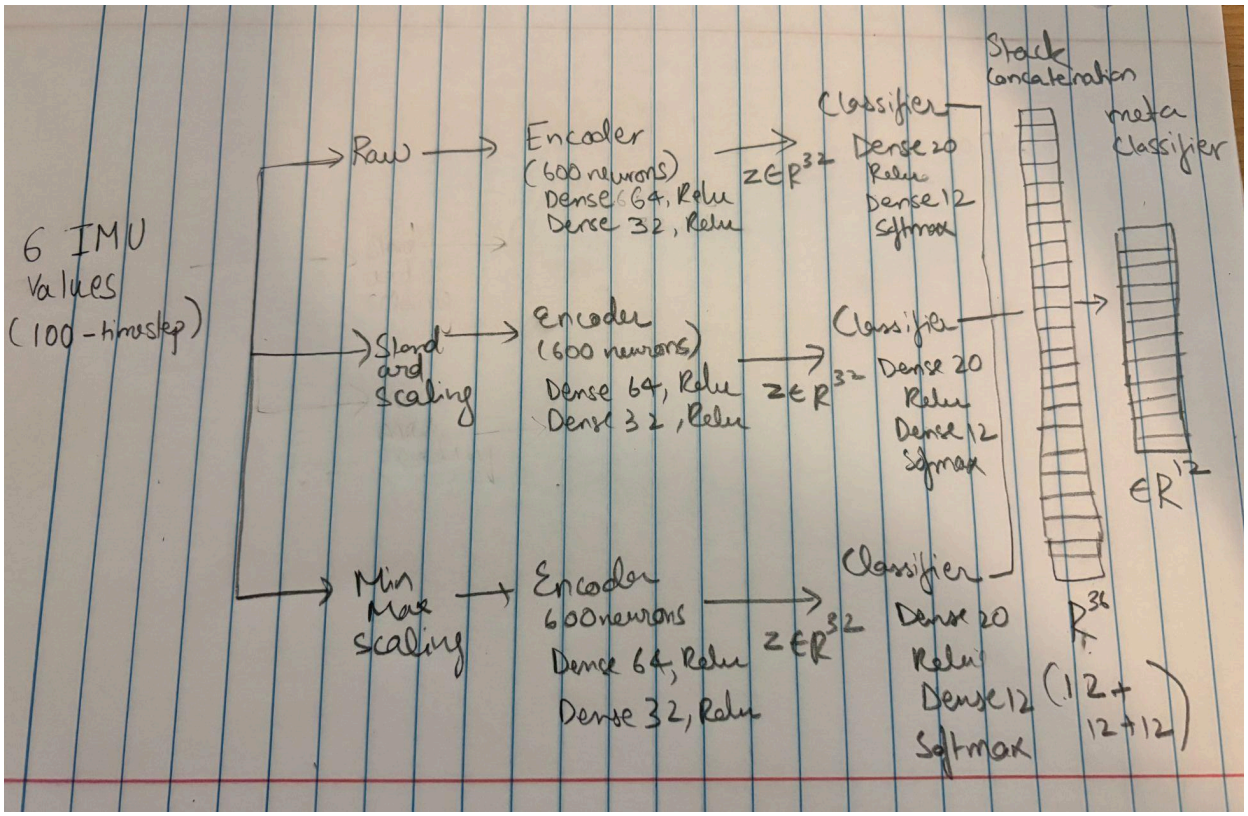
Draw a complete diagram of the full ensemble pipeline. Your diagram should include enough detail for someone to reconstruct the model architecture from your drawing.

Include the following details:

- **The input window size and number of input neurons. In this notebook, each input window has 100 time steps and 6 IMU features, so the flattened input dimension is 600.**

- **The three input branches: raw input, standard-scaled input, and min-max-scaled input.**
- **The encoder architecture in each branch, including the number of hidden layers, number of neurons in each layer, and activation function used in each layer.**
- **The classifier architecture in each branch, including the number of input neurons, hidden-layer neurons, output neurons, and activation functions.**
- **The number of models used in the ensemble and how they are connected.**
- **The size of each branch output and how the three outputs are concatenated before being passed into the stacked meta-classifier.**
- **The stacked meta-classifier architecture, including input size, hidden layer size, output size, and activation functions.**

A) Diagram of ensemble model



Question 2: Pruning and QAT methodology

Explain how this notebook compresses the models for TinyML deployment.

In your answer, describe:

- **How pruning is applied and what sparsity schedule is used.**

A)The notebook applies pruning with low magnitude, where the smallest and least important weights are gradually set to zero. It uses a Polynomial Decay sparsity schedule that starts at 10% sparsity and increases to 80% sparsity over 500 steps during 5 epochs of fine-tuning. This helps reduce the number of active weights while keeping important features of the CNN model.

- **Why are the pruning wrappers stripped before QAT?**

  A)After pruning, the model contains special pruning wrappers that manage masks and pruning steps. Before applying Quantization Aware Training, these wrappers are removed. This is necessary because QAT adds its own fake quantization wrappers, and both systems are not compatible together. Stripping pruning keeps the zeroed weights permanently in the model while allowing QAT to be applied cleanly.

- **How QAT is applied after pruning.**

  A)QAT is then applied to the stripped model using **quantize_model()**. Fake quantization operations are put into the network so the model learns how to work with int8 representation during training. The model is then fine-tuned again for 5 epochs so it becomes more robust to the rounding and reduced precision of int8 quantization.

- **Why does the notebook use an additional mask-enforcement step during and after QAT instead of using only a simple pruning-then-QAT pipeline.**

  A)During QAT, backpropagation can sometimes change pruned zero weights back to nonzero values. To prevent this, the notebook uses an additional mask-enforcement callback that reapplies the pruning masks after every batch and epoch. This guarantees that previously pruned weights remain zero throughout QAT. Without this step, much of the compression benefit from pruning would be lost.

- **How is the final model converted to an int8 TFLite model and why representative data is needed.**

  A)Finally, the model is converted into an int8 TensorFlow Lite model using `Optimize.DEFAULT` and `TFLITE_BUILTINS_INT8`. The converter also uses a representative dataset of sample inputs to calibrate the scaling and zero-point values for activations. This allows both weights and activations to be fully quantized into int8 format, improving efficiency and reducing memory usage.

- **How do pruning and int8 quantization help when deploying to a resource-constrained device such as the Arduino Nano 33 BLE Sense.**

  A)Pruning and int8 quantization are very useful for deployment on resource-constrained Arduino Nano 33 BLE Sense because they reduce model size, memory usage, and computation cost.

Question 3: Arduino deployment and real-world behavior

Deploy the quantized model using the provided Arduino sketch for this lab. Place the Arduino Nano 33 BLE Sense flat on a stable surface, open the serial monitor, and observe the predicted activity labels.

In your answer, include:

- **A screenshot of the serial monitor output.**
- **A short explanation of whether the output is acceptable for the resting board condition.**
- **Any issues you observe, such as unstable predictions, unexpected labels, sensor noise, scaling mismatch, or mismatch between the training data and the real board orientation.**
- **One or two practical changes that could improve deployment reliability.**

```
17:21:11.581 -> Final prediction: Sitting and relaxing
17:21:11.581 -> Class index: 1
17:21:11.581 -> Confidence score: 0.3555
17:21:11.581 -> ----------------------------------------
17:21:12.105 -> Samples collected: 25
17:21:12.635 -> Samples collected: 50
17:21:13.141 -> Samples collected: 75
17:21:13.690 -> Samples collected: 100
17:21:13.690 -> Window collected. Running ensemble inference...
17:21:13.690 -> Raw model scores: 0.0000, 0.2578, 0.1250, 0.0000, 0.0000, 0.0312, 0.0000, 0.0000, 0.0000, 0.0000, 0.5820, 0.0000
17:21:13.690 -> Standard-scaled model scores: 0.0000, 0.0000, 0.0156, 0.0000, 0.0000, 0.0000, 0.9766, 0.0000, 0.0039, 0.0000, 0.0000, 0.0039
17:21:13.728 -> Min-max-scaled model scores: 0.0000, 0.1094, 0.1328, 0.0000, 0.6289, 0.0781, 0.0000, 0.0469, 0.0000, 0.0000, 0.0000, 0.0000
17:21:13.728 -> Meta-classifier scores: 0.0000, 0.0039, 0.5586, 0.2344, 0.1602, 0.0000, 0.0078, 0.0000, 0.0000, 0.0000, 0.0156, 0.0117
17:21:13.728 -> ----------------------------------------
17:21:13.728 -> Final prediction: Lying down
17:21:13.728 -> Class index: 2
17:21:13.728 -> Confidence score: 0.5586
17:21:13.728 -> ----------------------------------------
17:21:14.272 -> Samples collected: 25
17:21:14.805 -> Samples collected: 50
17:21:15.299 -> Samples collected: 75
```

A)When the board was flat and stationary, the model mostly predicted Sitting and relaxing and Lying down consistently. This output is acceptable because the board was motionless, which produces sensor readings similar to low-activity postures such as sitting or lying down.

However, there were some issues during deployment. When the board was tilted, the model sometimes predicted Jump front and back even though the movement was more similar to Waist bends forward. In some stationary cases, the model also incorrectly predicted Walking. This could be due to discrepancies between how I did inference vs how the data was collected. Small accelerometer fluctuations/scale differences could also cause the model to confuse slow paced walking with stationary motion.

Two practical changes could improve deployment reliability:

1. Collecting more training data with different board orientations and stationary positions so the model learns a wider range of real-world conditions using the arduino board.
2. Different/More preprocessing of the raw sensor data to reduce noise.

Question 4: Deployment Behavior on the Arduino

After deploying the ensemble model on the Arduino Nano 33 BLE Sense / Sense Rev2, did you observe any unexpected prediction behavior? For example, did the model repeatedly predict the same class even when the board was moved differently?

A)Yes, after deploying the model on the Arduino, I observed unexpected prediction behavior. The model frequently predicted “Walking” even when the board was still. After introducing movement, the predictions changed, but often with a delay after several motions. The model also did not reliably display all 12 activity classes and most commonly predicted walking, lying down, and jumping.

Briefly explain why this may happen. In your answer, discuss possible causes such as differences between the original training dataset and the live Arduino sensor data, sensor placement, axis orientation, unit conversion, feature scaling, or mismatch between the motion patterns used during training and testing.

A)This may happen because the live sensor data collected from the Arduino differs from the original training dataset. The training data may have been collected with different motion intensity, sensor placement, or board orientation compared to my deployment. Differences in accelerometer orientation could also change the sensor patterns. Moreover, feature scaling or unit conversion mismatches between training and testing could affect prediction accuracy. Sensor noise and drifting values while the board is stationary may also resemble slow movement patterns such as walking. Finally, some motions performed during my testing may not closely match the exact motion patterns the model was trained on, leading to incorrect class predictions.